

Internet Technology (CMP 557)

Crash-Course Exam Guide — Units 1 to 4

Pokhara University · MSc CS · Compiled from syllabus + past-paper patterns

This guide covers everything your professor has taught so far (Units 1-4), in the depth needed for definition, comparison, and design-style exam questions — with diagrams for the topics that are easiest to lose marks on if you can't draw them from memory.

A short, honest add-on on Unit 5 (Security) is included at the end — you haven't covered it in class, but it showed up on the past paper and is conceptually easy to pick up fast.

How to Use This Guide Tonight

- **If you only have 2 hours:** Unit 2 (DNS + TCP/UDP + email protocols) and Unit 3 (HTTP/2 vs HTTPS vs QUIC, REST API) — these are the most 'question-friendly' topics and appear almost every year.
- **If you have 4+ hours:** add Unit 1 (ISP tiers/IXP diagram, routing, SDN) and Unit 4 (static vs dynamic, mirroring, load balancing).
- **If you have time left:** skim the Unit 5 quick-pass at the end — it's short and often worth easy marks.
- **For every essay question:** use the answer framework — Definition -> Components/How it works -> Why it matters (tie to security/scalability/performance) -> Example.
- **For 'design a system' questions:** state assumptions, sketch a simple diagram (client -> DNS -> LB -> servers -> DB), and justify each component in one line.

Coverage note: because attendance was irregular, this guide goes slightly deeper than a bare summary sheet — each unit includes the 'why it matters' reasoning examiners look for, not just definitions.

UNIT 1 — Overview of Internet Technology

1.1 Evolution of the Internet & Standards Bodies

The Internet evolved from **ARPANET** (1969, US Department of Defense — a fault-tolerant research network) through **NSFNET** (academic backbone, 1980s) to the **commercial Internet** of the 1990s, and today's mobile/IoT/Web 2.0 Internet.

- **IETF (Internet Engineering Task Force)**: develops and publishes RFCs — the technical protocol standards (TCP/IP, HTTP, DNS, BGP).
- **W3C (World Wide Web Consortium)**: standardizes web-facing technologies — HTML, CSS, accessibility guidelines.

Why it matters: without shared standards, equipment from different vendors and networks run by different organizations could not interoperate. Standards bodies keep the Internet open, interoperable, and able to scale to billions of devices.

1.3 Role of ISPs, NAPs, and IXPs

Term	What it is
ISP	Internet Service Provider — sells Internet access/connectivity to end users and businesses.
NAP	Network Access Point — older term for the physical infrastructure where ISPs interconnect.
IXP	Internet Exchange Point — modern equivalent of a NAP; a shared facility where multiple networks exchange traffic directly instead of routing through a third party, cutting latency and transit cost.

1.4 Tiered ISP Architecture

- **Tier 1**: owns global backbone infrastructure; does not pay anyone for transit; peers (settlement-free) with other Tier 1 networks.
- **Tier 2**: regional providers; peer with some networks but also buy transit from Tier 1 to reach the whole Internet.
- **Tier 3**: local/access ISPs; buy transit from Tier 2; connect end customers directly.

Tiered ISP Architecture with IXPs / NAPs

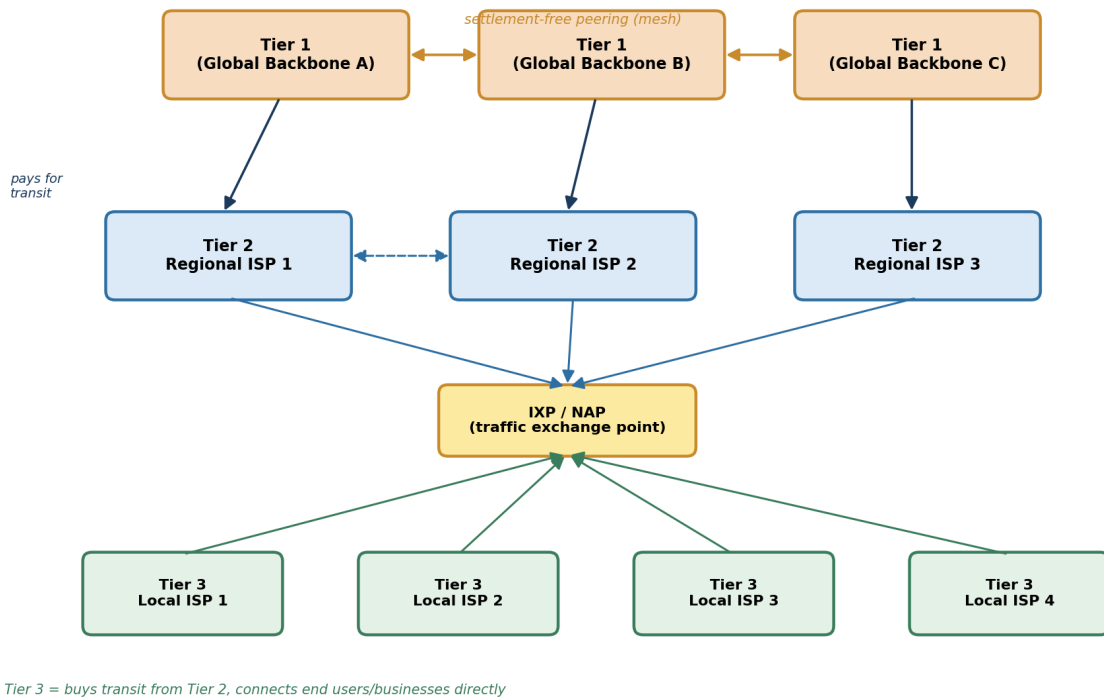


Fig 1.1 — Tiered ISP architecture: settlement-free peering at Tier 1, paid transit downward, IXPs enabling direct exchange.

1.5 TCP/IP & IP Addressing

TCP/IP layers: Application, Transport, Internet, Network Access (4-layer model, vs OSI's 7).

Version	Size	Approx. addresses	Example
IPv4	32-bit	~4.3 billion	192.168.1.1
IPv6	128-bit	~3.4 x 10 ³⁸ (effectively unlimited)	2001:0db8::1

IPv6 exists mainly to solve **IPv4 address exhaustion** — as devices multiplied (mobile, IoT), 32-bit space ran out.

1.6 Routing Protocols

- **Static routing:** manually configured routes; simple, low overhead, but doesn't adapt to link failures/topology changes — fine for small/stable networks.
- **Dynamic routing:** routers exchange routing info automatically (e.g. **OSPF**, **RIP**) and adapt to changes — needed for larger, changing networks.
- **BGP (Border Gateway Protocol):** the routing protocol *between* autonomous systems (i.e. between ISPs) — the reason the 'network of networks' can route traffic globally. It's what lets Tier 1/2/3 ISPs actually exchange reachability information.

1.7 Software-Defined Networking (SDN)

SDN separates the **control plane** (decides how traffic should be routed) from the **data plane** (actually forwards packets). A centralized software controller manages network behaviour programmatically, instead of configuring each hardware router/switch individually — giving far more flexibility, automation, and easier network-wide policy changes.

UNIT 2 — Internet Applications, Protocols and Services

2.1 Transport Layer: TCP vs UDP

Feature	TCP	UDP
Connection	Connection-oriented (3-way handshake)	Connectionless
Reliability	Reliable — acknowledgments + retransmission	Unreliable — no delivery guarantee
Order	Guarantees order	No order guarantee
Speed	Slower (overhead)	Faster (minimal overhead)
Use cases	HTTP, email, file transfer	VoIP, video streaming, DNS queries

TCP (connection-oriented) vs UDP (connectionless)

TCP: 3-way handshake

UDP: fire-and-forget

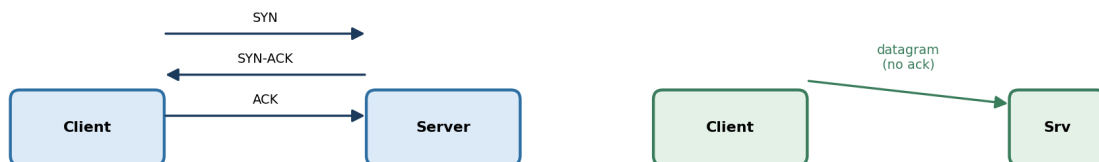


Fig 2.1 — TCP's handshake overhead vs UDP's fire-and-forget delivery.

Why UDP for VoIP (classic exam question): real-time voice needs speed over perfection. A brief lost/late packet just causes a tiny silence gap — barely noticeable. TCP's retransmission would instead cause noticeable lag, which is worse for a live conversation than the occasional glitch. Loss is instead handled by **jitter buffers**, **forward error correction (FEC)**, and loss-tolerant codecs — not retransmission.

2.2 Domain Name System (DNS)

DNS translates human-readable names (google.com) into IP addresses, structured as a hierarchy:

- **Root servers** — top of the hierarchy, point to TLD servers.
- **TLD servers** — handle .com, .org, .np, etc.
- **Authoritative name servers** — hold the actual records for a specific domain.
- **Recursive resolver** — queries the hierarchy on behalf of the client and returns the final answer.

DNS Resolution Process

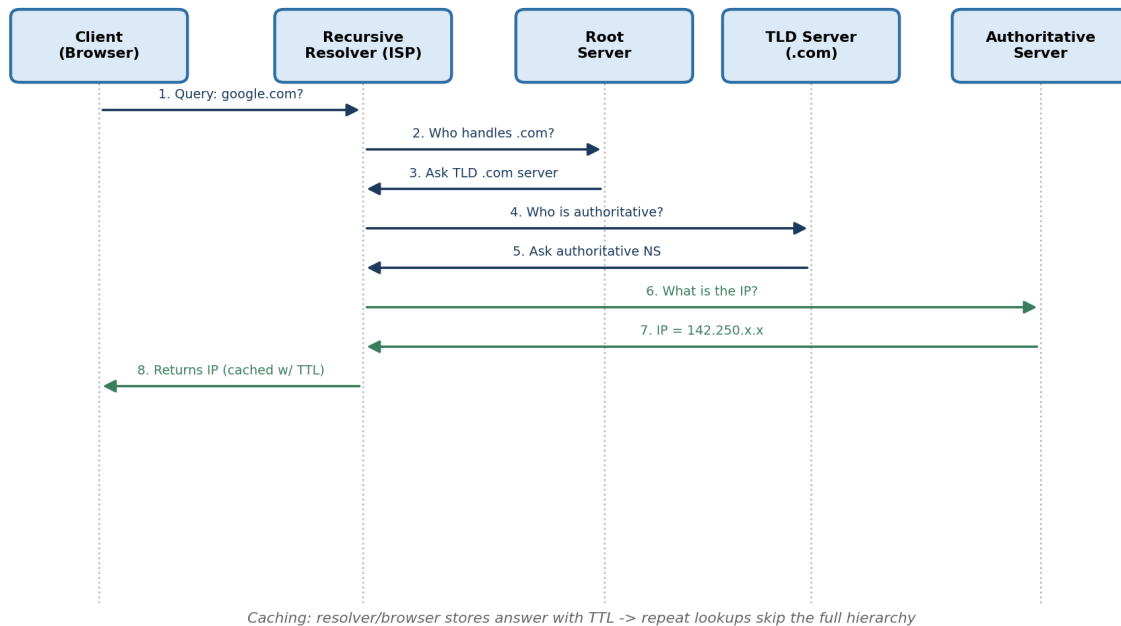


Fig 2.2 — Full DNS resolution path from client to authoritative server and back.

Caching: resolvers and browsers store answers with a **TTL (time-to-live)**, so repeat lookups skip the full hierarchy — this means faster resolution and less load on root/TLD servers.

2.3 Email Protocols

Protocol	Role
SMTP	Sends/relays outgoing mail (client-to-server and server-to-server).
POP3	Downloads mail to one device; typically removes it from the server afterward.
IMAP	Keeps mail on the server and syncs across multiple devices.

2.4 – 2.5 FTP/SFTP, Cloud Storage, VoIP, FoIP

- **FTP:** transfers files; unencrypted (credentials and data sent in plaintext).
- **SFTP:** FTP over SSH; encrypted, so it's the secure choice for the same job.
- **VoIP:** Voice over IP, real-time calls carried over UDP-based protocols — **RTP** carries the actual media, **SIP** handles call signaling (setup/teardown).
- **FoIP:** Fax over IP — carrying fax transmissions over an IP network instead of the traditional phone line.

2.6 – 2.7 Web Services, APIs, and CDN

CDN (Content Delivery Network): a set of geographically distributed servers that cache content closer to end users. Benefits: lower latency (shorter physical distance to travel) and reduced load on the origin server (most requests get served from the edge cache).

UNIT 3 — Web Technologies and Standards

3.1 Client-Server Architecture

The client (browser/app) sends requests; the server hosts resources/applications, processes the request, and returns a response. The client is responsible for rendering/consuming the result.

3.2 HTTP/1.1 vs HTTP/2.0 vs HTTPS vs QUIC (HTTP/3)

Version	Key feature	Limitation solved
HTTP/1.1	Text-based; one request per connection (head-of-line blocking even with pipelining)	—
HTTP/2.0	Multiplexing (many requests over one connection), header compression, server push	HTTP/1.1's connection overhead & app-layer head-of-line blocking
HTTPS	HTTP + TLS/SSL encryption	Security/eavesdropping — not a speed fix, can layer onto any version
QUIC / HTTP/3	Built on UDP; multiplexing without TCP's head-of-line blocking; 0-RTT handshake	TCP-level head-of-line blocking, which even HTTP/2 couldn't fix

Evolution: HTTP/1.1 -> HTTP/2 -> HTTP/3 (QUIC) | HTTPS = security layer

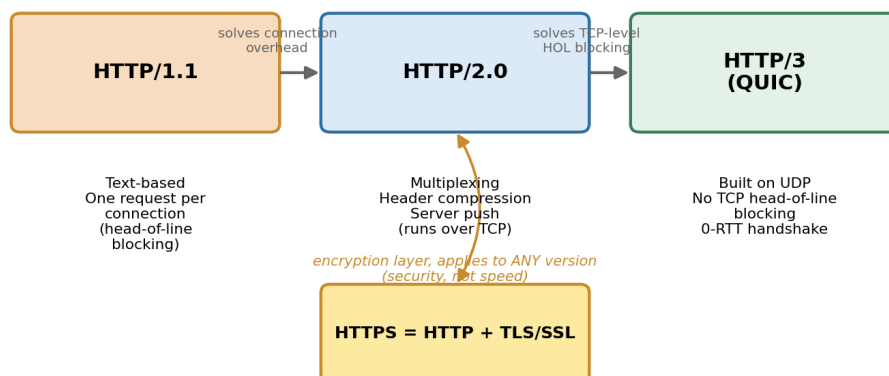


Fig 3.1 — HTTP/1.1 to HTTP/3 progression, with HTTPS as an orthogonal security layer.

'Which is better and why' answer frame: they solve different problems — HTTP/2 = efficiency, HTTPS = security, QUIC/HTTP3 = both speed and security on a UDP foundation. QUIC is generally the best choice for performance today (faster setup, no TCP-level blocking, encryption built in), but HTTPS is still the answer whenever the question is specifically about security.

3.3 RESTful APIs

REST uses standard HTTP methods to act on resources. Worked example — a library system:

Method + endpoint	Action
GET /books	List all books
GET /books/{id}	Get a specific book
POST /books	Add a new book
PUT /books/{id}	Update a book
DELETE /books/{id}	Remove a book

The client sends HTTP requests; the server responds with data (usually JSON) plus a status code (200 OK, 201 Created, 404 Not Found, etc.). If asked to **design** a REST API for a new scenario (e.g. a student management system), just map CRUD operations (Create/Read/Update/Delete) onto POST/GET/PUT/DELETE the same way.

3.4 – 3.5 MIME Types & Modern Web Architectures (mentioned in syllabus)

- **MIME types / content negotiation:** tell the browser/server what kind of data is being sent (text/html, application/json, image/png) so it's handled correctly.
- **Microservices:** an app is split into small, independently deployable services instead of one large monolith — improves scalability and fault isolation.
- **Serverless:** code runs in short-lived, provider-managed functions (no server management by the developer); scales automatically, billed per execution.

UNIT 4 — Web Server and Hosting Technologies

4.1 Client-Server Architecture (hosting context)

Same core idea as 3.1, applied to hosting: the server hosts the application/website and handles requests; common web server software includes **Apache**, **Nginx**, and **Microsoft IIS**.

4.2 Static vs Dynamic Content

Aspect	Static	Dynamic
Definition	Fixed content, same for every user	Generated per request — DB-driven, personalized
Speed	Fast — no processing	Slower — server-side processing
Example	University notice board	Online banking

- **University notice board -> static:** content changes infrequently, no personalization needed, fast delivery, low server cost.
- **Online banking -> dynamic:** needs personalized, real-time, secure data (balances, transactions) — requires server-side processing, authentication, and database queries.

4.3 Domain Name Registration and Hosting

A domain is registered through a registrar and pointed (via DNS records) at a hosting provider's server. Hosting types range from shared hosting (cheap, limited resources) to VPS, dedicated servers, and cloud hosting (elastic, pay-as-you-go).

4.4 Web Server Mirroring

Multiple identical copies of a server (mirrors) are placed in different locations. This improves **availability/fault tolerance** (if one mirror fails, others keep serving traffic) and reduces **latency** (users are routed to the nearest mirror).

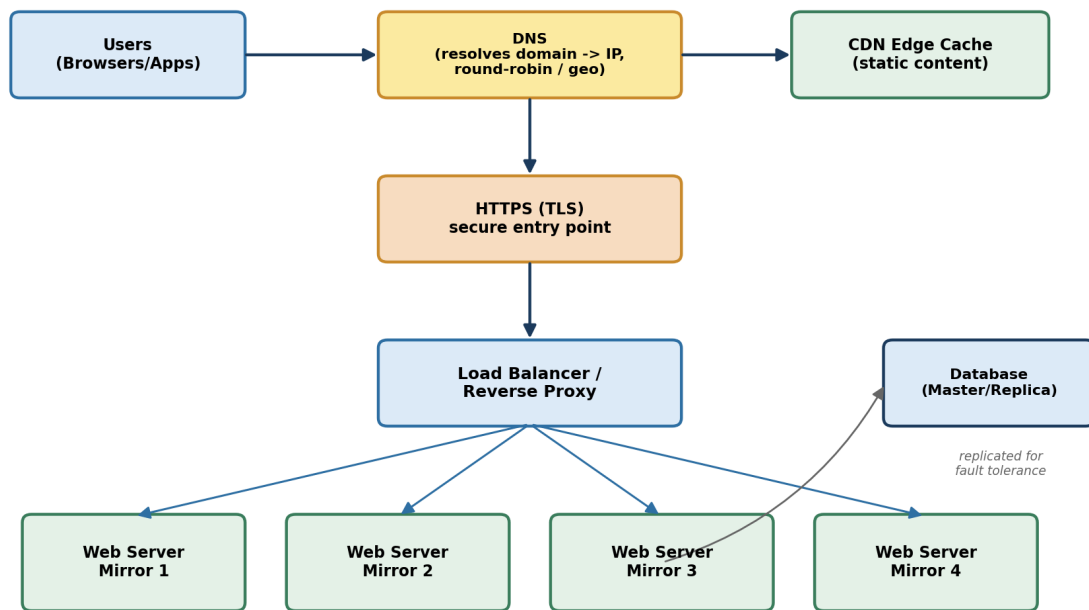
4.5 Proxy Load Balancing

A proxy/load balancer sits in front of multiple backend servers and distributes incoming requests across them. Contributes to:

- **Fault tolerance:** if one server fails, traffic is rerouted to healthy servers.
- **Scalability:** more capacity can be added by adding servers behind the balancer, without changing the client-facing address.

Worked Example: 'Design a scalable, secure web system' (typical design question)

Example Design: Scalable & Secure Web System



*Justification: mirrors + load balancer -> fault tolerance & scalability | CDN -> lower latency
HTTPS -> confidentiality/integrity | DNS -> distributes load across mirrors/regions*

Fig 4.1 — DNS routes users, CDN serves static assets, HTTPS secures the connection, a load balancer spreads load across mirrored servers, and the database is replicated.

How to answer this in the exam: state your assumptions (e.g. "expecting high, variable traffic; needs to be secure"), draw a simple version of the diagram above, then justify each block in one line each — DNS distributes users across regions, CDN cuts latency for static content, HTTPS protects data in transit, load balancer + mirrors give fault tolerance and scalability, and database replication protects against data loss. This single diagram answers most 'design/justify a system' questions in this course.

UNIT 5 (Quick Pass Only) — Internet Security

You haven't formally covered this in class, but it's on the syllabus, appeared on the past paper, and is intuitive enough to pick up quickly. Treat this as a light safety net, not a priority over Units 1-4.

5.1 – 5.2 Cryptography & Security Principles

- **Symmetric encryption:** same key encrypts and decrypts (fast, used for bulk data — e.g. AES).
- **Asymmetric encryption:** public/private key pair — public key encrypts or verifies, private key decrypts or signs (used to establish trust, e.g. RSA).
- **CIA triad** (core security principle): Confidentiality (only authorized parties can read data), Integrity (data isn't tampered with), Availability (systems stay accessible).

5.3 – 5.4 SSL/TLS, HTTPS, Digital Certificates

SSL/TLS encrypts the connection between client and server. A **digital certificate** (issued by a Certificate Authority) proves a server's identity and carries its public key, which the browser uses to set up the encrypted session — this is what makes HTTPS possible.

5.5 – 5.6 Web Application Security & Firewalls

- **OWASP Top 10** — the standard list of the most critical web app security risks (e.g. injection attacks, broken authentication, security misconfiguration).
- **Firewall:** filters traffic based on rules (allow/deny by IP, port, protocol).
- **IDS/IPS:** Intrusion Detection/Prevention Systems — detect (and, for IPS, actively block) suspicious traffic patterns.
- **VPN (secure tunneling):** creates an encrypted tunnel over a public network so traffic looks like it's on a private network.

If a question asks you to 'design a secure system,' the fast answer is: HTTPS for transport security, firewall + IDS/IPS at the network edge, strong authentication, and following OWASP guidance for the application layer.

Quick Answer Frameworks (Essay Questions)

- **"Explain X"** -> Definition -> Key components -> Why it matters / example.
- **"Compare X and Y"** -> Table of features -> 1-2 sentence verdict on when to use which.
- **"Design / justify a system"** -> State assumptions -> Diagram or steps -> Justify with pros/cons.
- **Always tie back to** security, scalability, or performance — examiners consistently reward candidates who connect a topic to one of these three themes.

Good luck tonight — prioritize Units 2 and 3 first, then circle back for Units 1 and 4.